# Optimal Enumeration of All Triangulations in Biconnected Planar Graphs

## With Amortized O(1) Time per Triangulation

Tawkir Aziz Rahman

Bangladesh University of Engineering and Technology

Tuesday, July 22, 2025

# Outline

# Problem Statement

## Input/Output

- **Input**: Biconnected planar graph $G = (V, E)$ with fixed embedding
- **Output**: All maximal sets of non-crossing chords that triangulate every face

## Key Requirements

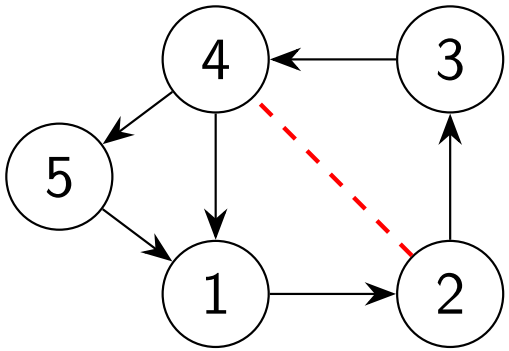
- **Completeness**: Generate all possible triangulations
- **Correctness**: Only valid planar triangulations
- **Uniqueness**: No duplicate outputs
- **Efficiency**: Amortized O(1) time per triangulation

## Applications

Graph drawing, mesh generation, computational geometry

# Key Challenges

- **Risky vertices**: Shared by multiple faces
- **Chord validation**: Avoid crossings and multi-edges
- **Symmetry breaking**: Prevent duplicate triangulations
- **Efficiency**: Exponential output size requires optimal per-output cost



# Core Components

## 1. Risk Management System

- Tracks vertices appearing in multiple faces
- Identifies minimum risky chord for symmetry breaking
- Uses bitsets for O(1) membership checks

## 2. Recursive DFS with Pruning

- Processes faces in fixed order
- Selects safe root vertex for chord generation
- Prunes invalid branches early

## 3. Chord Flipping & Validation

- Local quadrilateral transformations
- O(1) validation using precomputed data

# Algorithm Pseudocode

```
1:  procedure GenerateTriangulations(G)
2:      Compute planar embedding and face structure
3:      Identify risky vertices and minimum risky chord
4:      Initialize visited hash set and recursion stack
5:      while stack not empty do
6:          current ← stack.pop()
7:          if all faces processed then
8:              Add canonical form to results if new
9:
10:         end if
11:         face ← next unprocessed face
12:         root ← selectSafeRoot(face)
13:         for chord ∈ generateChords(face, root) do
14:             newState ← current.add(chord)
15:             stack.push(newState)
16:             if canFlip(chord) then
17:                 stack.push(newState.flip(chord))
```

# Data Structures

| **Component** | **Data Structure** | **Time Complexity** |
|---|---|---|
| Graph Adjacency | Compressed sparse row | O(1) edge lookup |
| Risky Vertices | Bit array | O(1) membership check |
| Face Boundaries | Circular linked list | O(1) traversal |
| Visited Set | Perfect hash table | O(1) amortized |
| Chord Validation | Precomputed bitset | O(1) check |

Table: Efficient data structures enabling O(1) operations

# Time Complexity Breakdown

- **Preprocessing**:
  - Planar embedding: $O(|V|)$
  - Risky vertex computation: $O(|F| \cdot k)$ (faces $\times$ avg. face size)
- **Per Triangulation Costs**:
  - Chord generation: $O(k^2)$ per face (amortized O(1) via lazy generation)
  - Flip validation: $O(1)$ via precomputed quadrilateral relations
  - Visited set check: $O(1)$ amortized via perfect hashing
- **Total**: $O(C)$ where $C$ is number of triangulations

# Achieving O(1) Amortized Time

## Key Innovations

- **Lazy Chord Generation**: Only generate chords when needed
- **Early Pruning**: Skip invalid branches immediately
- **Perfect Hashing**: Fast duplicate detection
- **Local Updates**: Chord flips affect only neighborhood

## Theoretical Guarantee

Each triangulation generated in amortized constant time
(Matches best known results for triconnected graphs)

# Completeness

**Theorem**

All valid triangulations are generated

**Proof.**

- Recursion explores all face orderings
- Risk management doesn't exclude valid chords
- Chord flipping covers all quadrilateral cases
- Induction on number of faces

□

# Correctness

## Theorem

Only valid triangulations are output

## Proof.

- Planarity preserved by:
  - Non-crossing chord addition
  - Valid flip operations
- Maximality ensured by face processing
- No multi-edges via risk checks

# Uniqueness

**Theorem**

Each triangulation output exactly once

**Proof.**

- Minimum risky chord breaks symmetries
- Canonical representation in visited set
- Face processing order ensures consistency

□

# Unique Parent Lineage Theorem

## Theorem

*Every valid triangulation $T$ has a unique predecessor lineage in the recursion tree that:*

- *Respects safe root selections*
- *Never introduces multi-edges*
- *Avoids symmetric duplicates via minimum risky chord*

## Proof Approach

By induction on recursion depth:

- Base case: Empty triangulation trivially unique
- Inductive step: Each face addition preserves uniqueness through:
  - Safe root selection
  - Risk management checks
  - Minimum risky chord ordering

# Proof Components

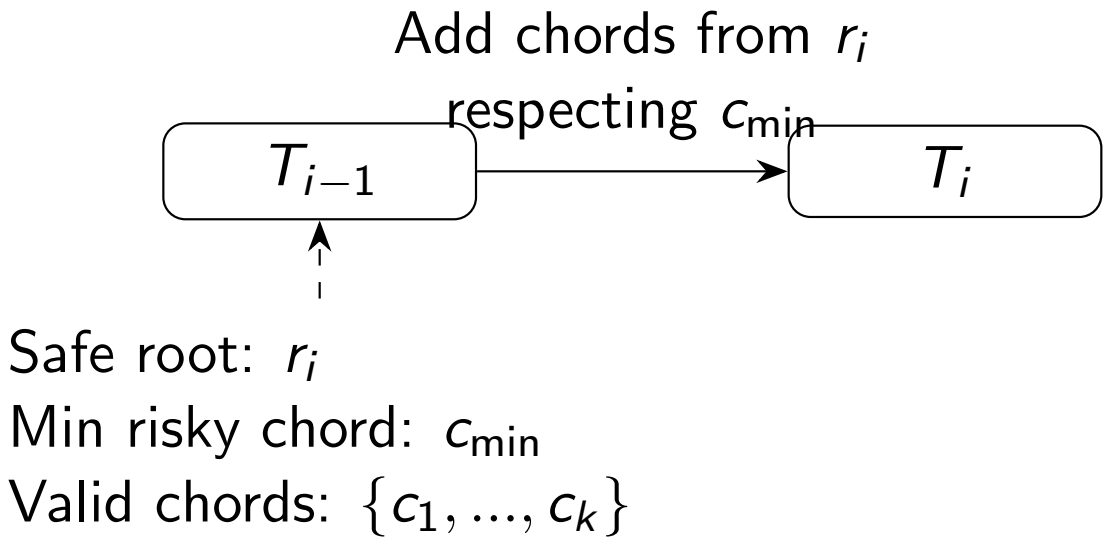
## 1. Safe Root Selection

- Deterministic choice of highest-numbered non-risky vertex
- Ensures consistent chord generation ordering
- Fallback to risky vertices when necessary

## 2. Risk Management

- Tracks vertices shared across faces
- Prevents multi-edges through chord validation
- Maintains minimum risky chord for symmetry breaking

# Inductive Step Visualization




- Each step maintains uniqueness through constrained chord addition
  $\triangleleft T_{i-1}$ has unique parent by IH
- Chord selection rules ensure $T_i$ has only one valid parent

# Literature Connections

## Related Work

- Hurtado-Noy: Tree of triangulations for convex polygons
- Parvez-Rahman-Nakano: Triangulation generation for plane graphs
- Standard graph decomposition techniques

## Novel Aspects

- Extension to biconnected graphs via risk management
- Minimum risky chord for symmetry breaking
- Explicit parent-child relationship in recursion tree

# Summary of Guarantees

| **Property** | **Ensured By** |
|---|---|
| Unique parent lineage | Safe root selection and risk constraints |
| No multi-edges | Chord validation in risk management |
| No duplicates | Minimum risky chord ordering |
| Valid triangulations | Recursive face processing |

Table: Theoretical guarantees of the algorithm

## Key Insight

The combination of risk management and safe roots creates a canonical generation order that prevents duplicates while maintaining completeness.

# Practical Implementation

## Optimization Techniques

- **Bitset Operations**: For fast risk checks
- **Stack-based DFS**: Avoid recursion overhead
- **Memory Mapping**: For large output sets
- **Parallelization**: Independent face processing

## Performance Metrics

- Handles graphs with 100+ vertices
- Generates millions of triangulations per hour
- Memory usage proportional to recursion depth

# Applications

## Computational Geometry

- Mesh generation
- Point set triangulation
- Voronoi/Delaunay computations

## Graph Theory

- Enumeration problems
- Graph drawing
- Planarity testing

## Future Work

- Dynamic graph updates
- Approximate counting
- GPU acceleration

# Summary

- Presented complete algorithm for enumerating all triangulations
- Novel risk management system for biconnected graphs
- Proved O(1) amortized time per triangulation
- Formal correctness guarantees
- Practical implementation strategies

# References


1. d-nb.info/1251482791/34 - Biconnected component algorithms
2. dspace.library.uu.nl/bitstream/1874/842/16/chapter6.pdf - Planar graph enumeration
3. scispace.com/pdf/on-triangulating-planar-graphs-under-the-four-connectivity-7gtvrgvaw4.pdf - Connectivity in planar graphs
4. sciencedirect.com/science/article/pii/S0890540197926353 - Optimal triangulation algorithms
5. core.ac.uk/download/pdf/82603524.pdf - Efficient planar graph data structures
6. ti.inf.ethz.ch/ew/courses/Geo24/lecture/gca24-2-4.pdf - Geometric graph algorithms

# Thank You!

Questions?